University of Toronto Scarborough

CSCC69

Winter 2022

Final Exam

First Name: _____

Last Name: _____

Student Number: _____

_____ @mail.utoronto.ca

Do not turn this page until you have received the signal to start.

In the meantime, fill out the identification section above and read the instructions below carefully.

This 3 hours exam contains 16 pages (including this cover page) and 6 parts. Check to see if any pages are missing. Enter all requested information on the top of this page, and put your initials on the top of every page, in case the pages become separated.

You may **not** use notes, or any calculator on this exam.

Do not write anything in the table below.

Part:	1	2	3	4	5	6	Total
Points:	16	20	25	27	12	6	106
Score:							

1. **Part 1 - Multithreading**

   Here is a new test case for Pintos project 1:

```
void main (void) {
  struct lock lock;
  lock_init (&lock);

  msg ("main tries to get the lock (priority %d)", thread_get_priority ());
  lock_acquire (&lock);

  msg ("main creates thread1 (priority %d)", thread_get_priority ());
  thread_create ("thread1", PRI_DEFAULT + 1, thread1, &lock);

  msg ("main creates thread2 (priority %d)", thread_get_priority ());
  thread_create ("thread2", PRI_DEFAULT + 2, thread2, &lock);

  msg ("main releases the lock (priority %d)", thread_get_priority ());
  lock_release (&lock);

  msg ("main is done (priority %d)", thread_get_priority ());
}

static void thread1 (void *lock_) {
  struct lock *lock = lock_;

  msg ("thread1 tries to get the lock (priority %d)", thread_get_priority ());
  lock_acquire (lock);

  msg ("thread1 releases the lock (priority %d)", thread_get_priority ());
  lock_release (lock);

  msg ("thread1 is done (priority %d)", thread_get_priority ());
}

static void thread2 (void *lock_) {
  struct lock *lock = lock_;

  msg ("thread2 tries to get the lock (priority %d)", thread_get_priority ());
  lock_acquire (lock);

  msg ("thread2 releases the lock (priority %d)", thread_get_priority ());
  lock_release (lock);

  msg ("thread2 is done (priority %d)", thread_get_priority ());
}
```

(1.1) 16 points - **my short answer**

Base on your understanding of the "priority scheduling" with priority donation. In which order the 11 messages will be printed on the console when the program is executed? In this question, assume that Multilevel Feedback Queue scheduler (MLFQ) is disabled and that `thread_get_priority` returns the effective thread priority taking into account priority donations.

1. `main tries to get the lock (priority 31)`
2. `main creates thread1 (priority 31)`
3. `thread1 tries to get the lock (priority 32)`
4. _____
5. _____
6. _____
7. _____
8. _____
9. _____
10. _____
11. _____

2. **Part 2 - System Calls**

Here is a new test case for Pintos project 2:

```
#include "tests/lib.h"
int main (int argc, char *argv[]) {
  msg ("Hello World!");
  return 0;
}
```

(2.1) 20 points - **my short answer**

Once project 2 is fully implemented, what happens when the function `msg("Hello World!");` is executed? No need for specific code but describe in 10 steps or less how the library is called, how the system call works, how the message is printed and how the value is returned to the user program.

1. _____

_____

2. _____

_____

3. _____

_____

4. _____

_____

5. _____

_____

6. _____

_____

7. _____

_____

8. _____

_____

9. _____

_____

10. _____

_____

3. **Part 3 - Virtual Memory**

(3.1) 10 points - **my short answer**

Once project 3 is fully implemented, what happens when a page is accessed but its corresponding frame is no longer in memory because it was swapped? No need for specific code but describe in 10 steps or less how the page read is resolved. Assume that this page was not allocated to a memory-mapped file.

1. _____

2. _____

3. _____

4. _____

5. _____

6. _____

7. _____

8. _____

9. _____

10. _____

In this exercise, we are going to test your understanding of three page replacement algorithms: FIFO, LRU, and Second Chance

Considering a system with four physical memory frames (that are initially empty) and given a page access sequence, you are tasked to show which references cause page faults and show which pages are in which frames after each page access.

To do so, complete all three tables below with the appropriate values for rows 2 to 11. Each row contains:

- the page accessed
- whether this access causes a page fault
- which frame is evicted if any (leave empty if none is evicted)
- which pages are in which frames for frame 0, frame 1, frame 2 and frame 3

In each table, the first 3 rows have been fully completed already.

(3.2) 5 points - **my short answer**

Complete the table for the FIFO (First In, First Out) algorithm.

FIFO (First In, First Out)						
Page Access	Page fault (yes/no)	Frame evicted	Frame 0	Frame 1	Frame 2	Frame 3
1	yes		1			
2	yes		1	2		
3	yes		1	2	3	
4						
1						
2						
5						
1						
2						
3						
4						
5						

(3.3) 5 points - **my short answer**

Complete the table for the LRU (Last Recently Used) algorithm.

LRU (Last Recently Used)						
Page Access	Page fault (yes/no)	Frame evicted	Frame 0	Frame 1	Frame 2	Frame 3
1	yes		1			
2	yes		1	2		
3	yes		1	2	3	
4						
1						
2						
5						
1						
2						
3						
4						
5						

(3.4) 5 points - **my short answer**

Complete the table for the Second Chance algorithm.

Second Chance						
Page Access	Page fault (yes/no)	Frame evicted	Frame 0	Frame 1	Frame 2	Frame 3
1	yes		1			
2	yes		1	2		
3	yes		1	2	3	
4						
1						
2						
5						
1						
2						
3						
4						
5						

4. **Part 4 - File System**

Given Pintos with fully implemented memory-mapped files (project 3) and buffer cache (project 4), let us assume that we run Pintos with two concurrent programs A and B that both access a file `data.bin` that contains 200 integers (of 4 bytes each).

Program A overwrites all integers in `data.bin` with the value 0

```
int main (int argc, const char *argv[]){
  int fd;
  int i;
  int v=0;

  fd = open ("data.bin")
  for (i=0; i<200; i++){
      write (fd, &v, 4);
  }
  close (fd);
}
```

While program B reads and prints each integer from `data.bin`

```
int main (int argc, const char *argv[]){
  int fd;
  int i;
  int v;

  fd = open ("data.bin")
  for (i=0; i<200; i++){
      read (fd, &v, 4);
      msg ("%d", v);
  }
  close (fd);
}
```

In this question, we are interested in how the data from `data.bin` is read and written the file system and the buffer cache during the execution. For each execution step below, explain what is happening in the system. Please be specific in your answers by describing:

- from where the data is read to where it is written (file system or the buffer cache)
- what and how much data is read/written
- a brief justification of why this operation is happening

We can assume that there is a context switch after each execution step and that the system has enough memory allocated to buffer cache (no replacement).

(4.1) 3 points - **my long answer**

1. Thread A starts and opens the file `data.bin`

(4.2) 3 points - **my long answer**

2. Thread B starts and opens the file `data.bin`

(4.3) 3 points - **my long answer**

3. Thread A goes through the first 100 integers (the first 100 loop iterations)

(4.4) 3 points - **my long answer**

4. Thread B goes through the first 100 integers

(4.5) 3 points - **my long answer**

5. Thread A goes through the last 100 integers

(4.6) 3 points - **my long answer**

6. Thread B goes through the last 100 integers

(4.7) 3 points - **my long answer**

7. Thread A closes the file and exits

(4.8) 3 points - **my long answer**

8. Thread B closes the file and exits

(4.9) 3 points - **my long answer**

In the end, what are the values printed by program B on the terminal?

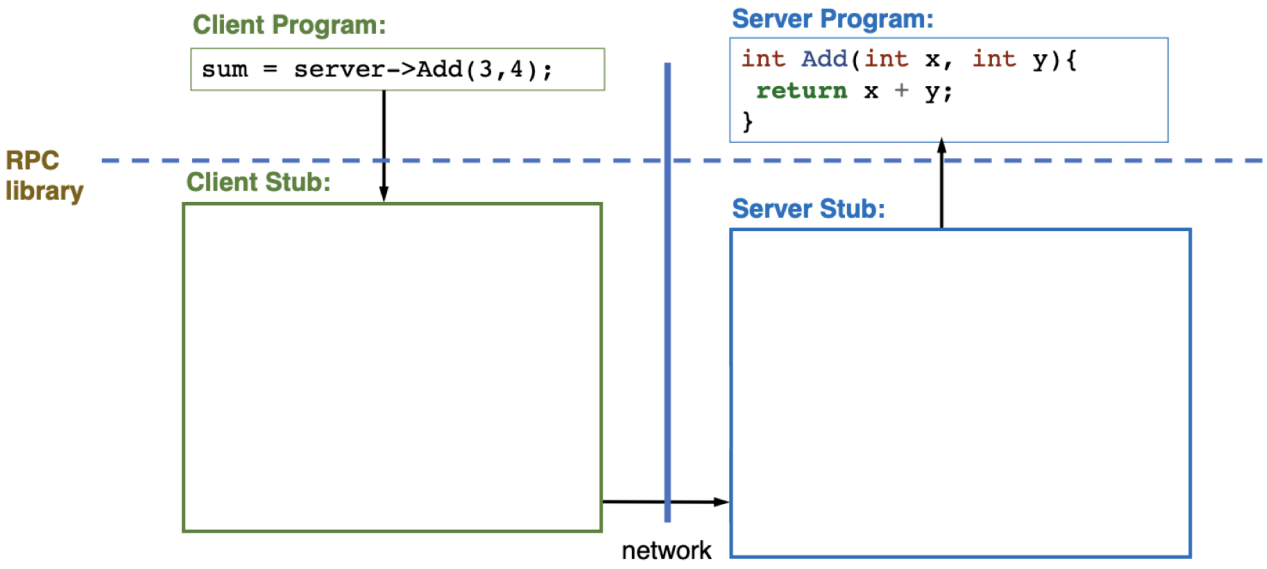
5. **Part 5 - RPC**

(5.1) 2 points - **my short answer**

RPC is the acronym for R_____ P_____ C_____

(5.2) 6 points - **my long answer**

When calling an RPC, explain what is happening in the client stub and the server stub. Please write your answer for each stub inside their respective rectangles in the image below.



On the client side, the RPC library waits for response for time T. If none arrives, it will re-send the request, possibly few times before raising an error. However, the client does not known why the request failed in the first place. If the request never reached the server, sending it again is useful. However, if the request did reached the server and has been executed the first time, it might be not desirable to send the same request again especially if that procedure has some side effects (i.e it modifies the state of the server). To fix this issue, RPC uses the `at-most-once` failure semantics in which the client includes an unique ID with each request and uses the same ID for re-send.

(5.3) 4 points - **my long answer**

Explain what the server does with this ID and how it fixes the problem.

6. **Bonus**

There are no bad answers to the questions below. You will get points if the instructor believes you genuinely made some efforts to provide an answer. Keep in mind that quantity does not beat quality.

(6.1) 2 points - **my long answer**

What are the strongest features of this course and of my teaching? In other words, what contributes most to your learning?

(6.2) 2 points - **my long answer**

What specific suggestions do you have for changes that I can make to improve the course or how it is taught?

(6.3) 2 points - **my long answer**

Share something with the course staff

Thank you for this great semester and have a good summer!